

AgentMatrix 部署文档与安装说明

多智能体动态协同与国产算力优化平台

版本: v0.1.0 | 日期: 2026-05-17

源代码获取

本项目源代码托管于 GitHub，请通过以下方式获取完整源代码：

克隆仓库

```
git clone https://github.com/han1326189481/AgentMatrix.git
cd AgentMatrix
```

GitHub 仓库地址

项目	地址
仓库首页	https://github.com/han1326189481/AgentMatrix
Git 克隆地址	<code>https://github.com/han1326189481/AgentMatrix.git</code>
SSH 克隆地址	<code>git@github.com:han1326189481/AgentMatrix.git</code>

提示: 建议使用 `git clone` 方式下载源代码，以确保获取完整的项目结构和版本历史。
下载后请先阅读本文档完成环境配置和安装。

一、项目技术概况

1.1 开发语言

层级	语言	版本	框架/运行时
后端	Python	3.12+	FastAPI + Uvicorn
前端	TypeScript	5.3+	Next.js 14 + React 18
样式	CSS / TailwindCSS	3.4	PostCSS
脚本	PowerShell / Bash	-	启动/停止脚本

1.2 调用的模型清单

本地模型 (通过 Ollama 运行)

模型名称	参数量	大小	用途
Qwen2.5 (qwen2.5:1.5b)	1.5B	~986 MB	Knowledge、Summary、Writer、Judge、Result Agent 的默认推理模型
Phi-4 Mini (phi4-mini:3.8b)	3.8B	~2.5 GB	Review Agent 专用推理模型 (需要更强逻辑推理能力)

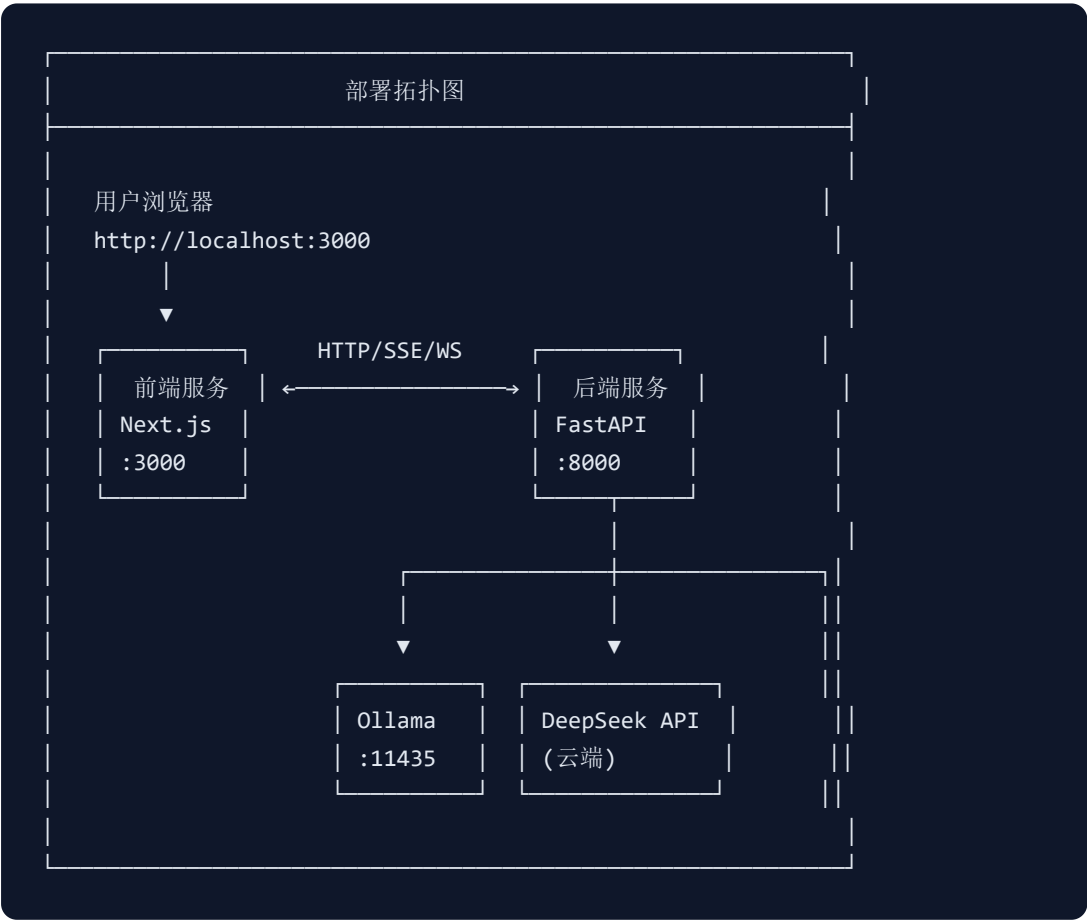
云端 API 模型

服务商	模型	用途
DeepSeek	deepseek-r1-distill	复杂任务云端增强推理 (默认云端模型)
DeepSeek	deepseek-v4-flash	高速云端推理 (可选)
DeepSeek	deepseek-chat	通用对话 (可选)
DeepSeek	deepseek-r1	深度推理 (可选)

服务商	模型	用途
OpenAI	<code>gpt-4o</code> / <code>gpt-4-turbo</code> / <code>gpt-3.5-turbo</code>	可选云端模型（需配置 API Key）
Anthropic	<code>claude-3-5-sonnet</code> / <code>claude-3-opus</code> / <code>claude-3-sonnet</code>	可选云端模型（需配置 API Key）
Google	<code>gemini-pro</code> / <code>gemini-1.5-pro</code> / <code>gemini-1.5-flash</code>	可选云端模型（需配置 API Key）

模型选择策略: Judge Agent 根据任务复杂度自动决定使用本地模型还是云端模型。复杂度 < 0.65 的任务由本地 Ollama 模型处理，复杂度 >= 0.65 的复杂任务自动升级至云端 DeepSeek API。云端模型可在前端「系统设置→添加云端模型」中自由切换。

二、部署架构概览



三、环境要求

3.1 硬件要求

环境	最低配置	推荐配置
开发环境	CPU 4核, 内存 8GB, 磁盘 20GB	CPU 8核, 内存 16GB, 磁盘 50GB
生产环境	CPU 4核, 内存 16GB, 磁盘 50GB	CPU 8核, 内存 32GB, 磁盘 100GB
GPU (可选)	-	NVIDIA GPU 8GB+ VRAM (加速本地模型推理)

3.2 软件要求

软件	最低版本	检查命令	用途
Python	3.12.0+	<code>python --version</code>	后端运行环境
Node.js	20.0.0+	<code>node --version</code>	前端运行环境
npm	9.0.0+	<code>npm --version</code>	前端包管理
Ollama	0.23.0+	<code>ollama --version</code>	本地模型推理服务
Git	2.40.0+	<code>git --version</code>	代码版本管理

3.3 操作系统支持

操作系统	支持状态	架构	备注
Windows 10/11	✅ 完全支持	x86_64	PowerShell 5.1+
macOS 12+	✅ 完全支持	x86_64 / ARM64	原生终端

操作系统	支持状态	架构	备注
Ubuntu 22.04+	完全支持	x86_64 / ARM64	Bash, apt 包管理
统信UOS (桌面专业版)	完全支持	x86_64 / ARM64 (鲲鹏)	基于 Deepin, apt 包管理
统信UOS (服务器版)	完全支持	x86_64 / ARM64 (鲲鹏)	基于 Deepin, apt 包管理
麒麟OS (桌面版 V10)	完全支持	x86_64 / ARM64 (鲲鹏/飞腾)	基于 openEuler/Ubuntu, rpm 或 apt
麒麟OS (服务器版 V10)	完全支持	x86_64 / ARM64 (鲲鹏/飞腾)	基于 openEuler, rpm 包管理
CentOS 7 / RHEL 7+	支持	x86_64	yum/dnf 包管理

说明: 统信UOS 和麒麟OS 均为国产自主可控操作系统, 本项目已完成兼容性适配。部署方式与 Linux 基本一致, 仅在包管理器和基础环境安装方式上有细微差异, 详见下方各系统的安装指引。

四、安装基础环境

4.1 安装 Python 3.12+

Windows: 1. 访问 <https://www.python.org/downloads/> 2. 下载 Python 3.12 安装包 3. 安装时勾选 "Add Python to PATH" 4. 验证: `python --version`

Ubuntu / 统信UOS (桌面版):

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

统信UOS (服务器版) / 麒麟OS (桌面版 V10):

```
# 统信UOS 和麒麟OS 桌面版通常自带 Python 3.x
# 若版本不满足, 可从源码编译安装
sudo apt update
sudo apt install build-essential libssl-dev zlib1g-dev libbz2-dev libreadlin
e-dev libsqlite3-dev

# 下载 Python 3.12 源码
wget https://www.python.org/ftp/python/3.12.0/Python-3.12.0.tgz
tar -xzf Python-3.12.0.tgz
cd Python-3.12.0
./configure --enable-optimizations
make -j$(nproc)
sudo make altinstall

# 验证
python3.12 --version
```

麒麟OS (服务器版 V10) / CentOS / RHEL:

```
# 安装编译依赖
sudo dnf install -y gcc openssl-devel bzip2-devel libffi-devel zlib-devel re
adline-devel sqlite-devel

# 下载 Python 3.12 源码
wget https://www.python.org/ftp/python/3.12.0/Python-3.12.0.tgz
tar -xzf Python-3.12.0.tgz
cd Python-3.12.0
./configure --enable-optimizations
make -j$(nproc)
sudo make altinstall

# 验证
python3.12 --version
```

macOS:

```
brew install python@3.12
```

4.2 安装 Node.js 20+

Windows / macOS: 1. 访问 <https://nodejs.org/> 2. 下载 Node.js 20 LTS 版本 3. 验证: `node --version` 和 `npm --version`

Ubuntu / 统信UOS (桌面版):

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install nodejs
```

统信UOS (服务器版) / 麒麟OS (桌面版 V10):

```
# 方式一：使用 NodeSource (apt)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# 方式二：使用 nvm (推荐，更灵活)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | b
ash
source ~/.bashrc
nvm install 20
nvm use 20
```

麒麟OS (服务器版 V10) / CentOS / RHEL:

```
# 使用 nvm 安装 (推荐)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | b
ash
source ~/.bashrc
nvm install 20
nvm use 20

# 或使用 NodeSource (dnf)
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo dnf install -y nodejs
```

4.3 安装 Ollama

Windows: 1. 访问 <https://ollama.com/download/windows> 2. 下载并安装 3. 验证: `ollama --version`

Ubuntu / 统信UOS / 麒麟OS (基于 apt 的系统):

```
curl -fsSL https://ollama.com/install.sh | sh
```

麒麟OS (基于 rpm 的系统) / CentOS / RHEL:

由于 Ollama 官方暂未提供 rpm 包，建议使用以下两种方式之一：

```
# 方式一：使用官方 Linux 安装脚本 (通用性最好)
curl -fsSL https://ollama.com/install.sh | sh
```

```
# 方式二：手动下载二进制文件
# 下载适用于 Linux 的 Ollama 二进制文件
wget https://ollama.com/download/ollama-linux-amd64.tgz
sudo tar -C /usr -xzf ollama-linux-amd64.tgz

# 创建 systemd 服务
sudo tee /etc/systemd/system/ollama.service > /dev/null << 'EOF'
[Unit]
Description=Ollama Service
After=network.target

[Service]
Type=simple
User=root
ExecStart=/usr/bin/ollama serve
Restart=always
Environment="OLLAMA_HOST=0.0.0.0:11435"

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable ollama
sudo systemctl start ollama
```

注意: 在国产 ARM64 架构（鲲鹏 920、飞腾 S2500 等）上部署时，Ollama 和模型均提供原生 ARM64 支持，安装命令与 x86_64 一致。

macOS:

```
brew install ollama
```

五、安装步骤

5.1 获取源代码

```
# 从 GitHub 克隆项目
git clone https://github.com/han1326189481/AgentMatrix.git
cd AgentMatrix
```

5.2 配置 Ollama 并下载本地模型

Ollama 默认监听 `11434` 端口。本项目的默认配置使用 **11435** 端口（避免端口冲突）：

```
# Linux / macOS / 统信UOS / 麒麟OS
export OLLAMA_HOST="0.0.0.0:11435"
ollama serve

# Windows PowerShell
$env:OLLAMA_HOST="0.0.0.0:11435"
ollama serve
```

下载所需的本地模型（各约需 5-20 分钟）：

```
# 下载 Qwen2.5 1.5B 模型（~986MB）
ollama pull qwen2.5:1.5b

# 下载 Phi-4 Mini 3.8B 模型（~2.5GB）
ollama pull phi4-mini:3.8b

# 验证模型已安装
ollama list
```

预期输出：

NAME	ID	SIZE	MODIFIED
phi4-mini:3.8b	78fad5d182a7	2.5 GB	...
qwen2.5:1.5b	65ec06548149	986 MB	...

验证 Ollama 服务：

```
curl http://localhost:11435/api/tags
```

5.3 后端安装

Windows

```
cd backend

# 创建虚拟环境
python -m venv venv
```

```
# 激活虚拟环境（如果遇到执行策略限制，先运行：Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser）
.\venv\Scripts\Activate.ps1

# 安装核心依赖
pip install fastapi==0.110.0 uvicorn==0.29.0 pydantic==2.6.0
pip install python-dotenv==1.0.0 aiohttp==3.9.0 sqlalchemy==2.0.0
pip install loguru==0.7.0 websockets==12.0 psutil==5.9.0
pip install python-pptx==0.6.23 python-docx==1.1.0 pymdown-extensions==10.0.0

cd ..
```

Ubuntu / 统信UOS (桌面版) / macOS

```
cd backend

# 创建虚拟环境
python3 -m venv venv
source venv/bin/activate

# 安装依赖
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru websockets psutil
pip install python-pptx python-docx pymdown-extensions

cd ..
```

统信UOS (服务器版) / 麒麟OS (桌面版 V10)

```
cd backend

# 创建虚拟环境（注意：部分国产OS的 Python 命令可能为 python3.12）
python3.12 -m venv venv
source venv/bin/activate

# 安装依赖
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru websockets psutil
pip install python-pptx python-docx pymdown-extensions

cd ..
```

麒麟OS (服务器版 V10) / CentOS / RHEL

```
cd backend

# 使用 altinstall 安装的 Python 3.12
python3.12 -m venv venv
source venv/bin/activate

# 安装依赖
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru
websockets psutil
pip install python-pptx python-docx pymdown-extensions

cd ..
```

5.4 前端安装

所有操作系统:

```
cd frontend
npm install
cd ..
```

5.5 配置环境变量

后端配置

创建 `backend/.env` 文件:

Windows PowerShell:

```
@""
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=true
LOG_LEVEL=INFO
DATABASE_URL=sqlite:///./agentmatrix.db
OLLAMA_HOST=http://localhost:11435
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b
DEEPSEEK_API_KEY=
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-r1-distill
COMPLEXITY_THRESHOLD=0.65
```

```
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
"@ | Out-File -FilePath backend\.env -Encoding UTF8
```

Linux / macOS / 统信UOS / 麒麟OS:

```
cat > backend/.env << 'EOF'
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b
DEEPSEEK_API_KEY=
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-r1-distill
COMPLEXITY_THRESHOLD=0.65
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
EOF
```

说明: DeepSeek API Key 为可选项。不配置则所有任务由本地 Ollama 模型处理；配置后复杂任务将自动调用云端 DeepSeek 增强推理。

国产 OS 注意: 如果在服务器环境中部署，建议将 `ALLOWED_ORIGINS` 设置为实际的访问域名或 IP。

前端配置

创建 `frontend/.env.local` 文件:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000
```

六、启动服务

6.1 开发环境启动

需要同时启动三个终端窗口:

终端 1 - Ollama 服务:

```
ollama serve
```

终端 2 - 后端服务:

Windows:

```
cd backend
.\venv\Scripts\Activate.ps1
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Linux / macOS / 统信UOS / 麒麟OS:

```
cd backend
source venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

启动成功日志:

```
INFO:      Database tables created successfully
INFO:      Database initialized successfully
INFO:      Loaded knowledge base with 20 keywords
INFO:      All agents initialized successfully
INFO:      WebSocket manager initialized
INFO:      Application startup complete.
```

终端 3 - 前端服务:

```
cd frontend
npm run dev
```

启动成功日志:

```
▲ Next.js 14.1.0
- Local:      http://localhost:3000
✓ Ready in 5.4s
```

6.2 Windows 一键启动

```
# 在项目根目录运行
.\start.ps1
```

七、安装验证

7.1 验证 Ollama

```
ollama list
```

7.2 验证后端

```
curl http://localhost:8000/health  
# 预期: {"status":"healthy","agents":{"knowledge":{"status":"ready"},...}}
```

浏览器访问 <http://localhost:8000/docs> 查看 API 文档。

7.3 验证前端

浏览器访问 <http://localhost:3000>, 预期看到 AgentMatrix 主界面: - ☒ 顶部任务输入框 - ☒ 左侧 Agent 舰队列表 - ☒ 中间流水线可视化 - ☒ 右侧输出面板

7.4 端到端测试

在输入框输入 "你好, 请介绍一下你自己", 预期: 6 个 Agent 依次执行, 左侧"成本节省"指标增加, 右侧显示最终结果。

八、生产环境部署

8.1 生产模式启动

后端:

```
cd backend  
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

前端:

```
cd frontend
npm run build
npm start
```

8.2 使用 systemd (Linux / 统信UOS / 麒麟OS)

后端服务 (`/etc/systemd/system/agentmatrix-backend.service`):

```
[Unit]
Description=AgentMatrix Backend Service
After=network.target ollama.service

[Service]
Type=simple
User=agentmatrix
WorkingDirectory=/opt/agentmatrix/backend
Environment=PATH=/opt/agentmatrix/backend/venv/bin:/usr/bin
ExecStart=/opt/agentmatrix/backend/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

前端服务 (`/etc/systemd/system/agentmatrix-frontend.service`):

```
[Unit]
Description=AgentMatrix Frontend Service
After=network.target agentmatrix-backend.service

[Service]
Type=simple
User=agentmatrix
WorkingDirectory=/opt/agentmatrix/frontend
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable agentmatrix-backend agentmatrix-frontend
sudo systemctl start agentmatrix-backend agentmatrix-frontend
```

8.3 国产 OS 生产部署注意事项

统信UOS 服务器版

```
# 1. 确保防火墙开放端口
sudo ufw allow 3000/tcp
sudo ufw allow 8000/tcp

# 2. 若使用 SELinux (默认启用)
sudo semanage port -a -t http_port_t -p tcp 3000
sudo semanage port -a -t http_port_t -p tcp 8000

# 3. 设置为开机自启动
sudo systemctl enable ollama agentmatrix-backend agentmatrix-frontend
```

麒麟OS 服务器版 V10

```
# 1. 麒麟OS 使用 firewalld 管理防火墙
sudo firewall-cmd --permanent --add-port=3000/tcp
sudo firewall-cmd --permanent --add-port=8000/tcp
sudo firewall-cmd --reload

# 2. 若使用 SELinux
sudo semanage port -a -t http_port_t -p tcp 3000
sudo semanage port -a -t http_port_t -p tcp 8000

# 3. 设置开机自启
sudo systemctl enable ollama agentmatrix-backend agentmatrix-frontend
```

ARM64 架构 (鲲鹏 920 / 飞腾 S2500) 特殊说明

```
# 国产 ARM64 CPU 上部署时注意:
# 1. Python 建议从源码编译以充分利用 ARM 优化
# 2. Ollama 原生支持 ARM64, 直接使用官方脚本安装
# 3. Node.js 提供官方 ARM64 二进制包
# 验证架构
uname -m # 应输出: aarch64
```

8.4 使用 Docker

docker-compose.yml:

```

version: '3.8'

services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - OLLAMA_HOST=http://ollama:11434
      - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
    volumes:
      - ./backend/data:/app/data
    depends_on:
      - ollama
    restart: unless-stopped

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    environment:
      - NEXT_PUBLIC_API_URL=http://backend:8000
      - NEXT_PUBLIC_WS_URL=ws://backend:8000
    depends_on:
      - backend
    restart: unless-stopped

  ollama:
    image: ollama/ollama:latest
    ports:
      - "11435:11434"
    volumes:
      - ollama_data:/root/.ollama
    restart: unless-stopped

volumes:
  ollama_data:

```

```

# 启动全部服务
docker-compose up -d

# 下载模型到容器
docker-compose exec ollama ollama pull qwen2.5:1.5b
docker-compose exec ollama ollama pull phi4-mini:3.8b

```

8.5 Nginx 反向代理配置

```
server {
    listen 80;
    server_name agentmatrix.example.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api/ {
        proxy_pass http://localhost:8000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /ws {
        proxy_pass http://localhost:8000/ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

九、健康检查与监控

9.1 健康检查端点

端点	方法	用途
/health	GET	系统整体健康状态 + Agent 状态
/api/v1/metrics	GET	系统运行指标
/api/v1/chat/health	GET	聊天服务状态
/api/v1/workflow/cache/stats	GET	工作流缓存统计

9.2 系统监控指标

GET /api/v1/metrics 返回: - total_requests - 总请求数 - local_executions - 本地执行次数 - cloud_executions - 云端执行次数 - api_calls - API 调用次数 - cost_saved - 节省费用估算

9.3 日志配置

```
# backend/.env
LOG_LEVEL=INFO          # 开发: DEBUG, 生产: INFO 或 WARNING
LOG_FILE=logs/system.log
```

十、常见问题排查

Q1: PowerShell 无法激活虚拟环境?

错误: "无法加载文件 .\venv\Scripts\Activate.ps1, 因为在此系统上禁止运行脚本。"
 解决: 以管理员身份运行 PowerShell, 执行:
 Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Q2: pip install 报错?

解决:

1. 确保 Python >= 3.12: python --version
2. 更新 pip: python -m pip install --upgrade pip
3. 如果在公司网络, 配置代理: pip install --proxy=http://proxy:port ...

Q3: npm install 报错?

解决:

1. 删除 node_modules 和 package-lock.json
2. 重新安装: npm install
3. 或使用: npm install --legacy-peer-deps

Q4: Ollama 连接失败?

现象: 后端日志 "Failed to call Ollama: Connection refused"

解决:

1. 确认 Ollama 正在运行: `ollama list`
2. 检查端口号: Windows 项目配置 11435, Linux/macOS/国产OS 默认 11434
3. 测试连接: `curl http://localhost:11435/api/tags`
4. 修改 `backend/.env` 中的 `OLLAMA_HOST` 为正确端口

Q5: 前端页面空白?

解决:

1. 确认后端正在运行: `curl http://localhost:8000/health`
2. 检查 `frontend/.env.local` 配置
3. 打开浏览器开发者工具 (F12) 查看错误信息

Q6: 端口被占用?

现象: uvicorn 启动报 "Address already in use"

解决:

1. 查找占用进程: `netstat -ano | findstr :8000` (Windows) / `ss -tlnp | grep 8000` (Linux)
2. 修改端口: 在 `.env` 中设置 `SERVER_PORT=8001`

Q7: DeepSeek API 调用失败?

现象: "Error: DeepSeek API Key 未设置"

解决:

1. 访问 <https://platform.deepseek.com/> 获取 API Key
2. 添加到 `backend/.env` 中 `DEEPSEEK_API_KEY`
3. 不配置 API Key 则所有任务本地处理

Q8: 模型下载速度慢?

解决:

1. 使用代理加速下载
2. 仅下载需要的模型 (至少需要 `qwen2.5:1.5b`)
3. 可在另一台机器下载后拷贝模型文件

Q9: 统信UOS / 麒麟OS 上 Python 命令找不到?

现象: "python3: command not found"
解决:
统信UOS / 麒麟OS 桌面版可能使用 python3 别名
which python3 || which python3.12

创建软链接 (如果需要)
sudo ln -s /usr/bin/python3.12 /usr/bin/python3

Q10: 国产 ARM64 平台上安装依赖失败?

现象: pip 安装某些包时编译失败
解决:
1. 确保安装了 gcc 和开发库: sudo apt install build-essential python3-dev
2. 某些包可能尚未提供 ARM64 预编译 wheel, pip 会自动从源码编译
3. 如遇到特定包编译失败, 可尝试使用系统包管理器安装对应的 -dev 包

十一、安全注意事项

- 1. **API 密钥管理:** 不要将 .env 文件提交到版本控制。已加入 .gitignore。
- 2. **生产环境 CORS:** 修改 ALLOWED_ORIGINS 为实际前端域名, 不要使用 *。
- 3. **防火墙:** 生产环境应配置防火墙, 仅开放必要端口 (80/443)。
- 4. **HTTPS:** 生产环境应使用 SSL/TLS 加密通信。
- 5. **定期更新:** 定期更新 Ollama 模型和后端依赖, 获取安全补丁。
- 6. **国产 OS 安全:** 统信UOS 和麒麟OS 默认启用安全策略 (SELinux / AppArmor), 部署时如遇到权限问题, 请检查安全日志并配置相应策略。

十二、目录结构说明

```
AgentMatrix/  
├── backend/                # 后端服务 (Python + FastAPI)  
│   ├── agents/            # 6 个 Agent 实现  
│   │   ├── base/          # Agent 基类和注册中心  
│   │   ├── knowledge/     # Knowledge Agent (知识检索)  
│   │   └── summary/       # Summary Agent (需求摘要)
```

```

├── writer/           # Writer Agent (内容生成)
├── review/          # Review Agent (质量评审)
├── judge/           # Judge Agent (复杂度判断)
├── result/          # Result Agent (成果导出)
├── api/             # API 路由
├── v1/              # v1 版本 API
├── websocket/       # WebSocket 管理
├── core/            # 核心业务逻辑
├── workflow/        #  workflow编排
├── llm/             # LLM 客户端
├── dynamic_router/  # 动态算力路由
├── models/          # Pydantic 数据模型
├── prompts/         # Prompt 模板
├── knowledge/       # 知识库服务
├── config/          # 配置管理
├── services/        # 聚合服务层
├── app/             # 应用入口
├── main.py          # FastAPI 入口
├── config.py        # 全局配置
├── database.py      # 数据库初始化
├── tests/           # 测试

├── frontend/       # 前端服务 (TypeScript + Next.js)
├── src/
├── app/             # Next.js App Router
├── components/      # React 组件
├── layout/          # 主界面布局
├── stores/          # Zustand 状态管理
├── services/api/    # API 调用封装
├── types/           # TypeScript 类型定义
├── package.json     # 依赖清单
├── next.config.js   # Next.js 配置

├── configs/         # 配置文件
├── docs/            # 项目文档
├── scripts/         # 辅助脚本
├── start.ps1        # Windows 一键启动脚本
├── start.bat        # Windows 批处理启动脚本
├── stop.ps1         # Windows 停止脚本
├── README.md        # 项目说明

```

GitHub 仓库: <https://github.com/han1326189481/AgentMatrix>

文档版本: v1.0

最后更新: 2026-05-17